

Student Information

Name: Mujtaba Hejji Alnuwaysir

Student ID: 2240007941

Course: ARTI404 – Image Processing

Lab Session: Lab 4: Intensity Transformations and Filtering-Spatial Domain

Lab 4 Report: Intensity Transformations and Filtering-Spatial Domain

Objective

The main objective of this lab is to implement fundamental image processing algorithms, specifically focusing on intensity transformations and histogram processing techniques. Throughout the tasks, we applied contrast stretching, histogram equalization, and histogram matching to enhance image quality and adjust pixel intensity distributions using Python and the skimage library.

Task 1: Contrast Stretching (3rd to 80th percentiles)

Description

In this task, we performed contrast stretching on the 'moon' image. We calculated the 3rd and 80th percentiles of the original image's pixel intensities and rescaled them to span the full dynamic range.

Python Code

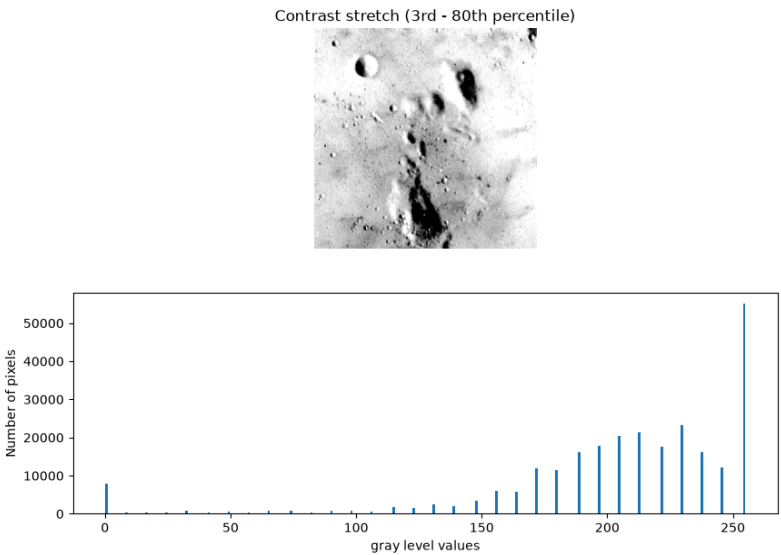
```
import matplotlib.pyplot as plt
import numpy as np
from skimage import data, exposure

img = data.moon()

def show(image, title, rng=(0, 255)):
    fig = plt.figure(figsize=(10, 7))
    fig.add_subplot(2, 1, 1)
    plt.imshow(image, cmap='gray')
    plt.axis('off')
    plt.title(title)
    fig.add_subplot(2, 1, 2)
    plt.hist(image.flat, bins=256, range=rng)
    plt.xlabel('gray level values')
    plt.ylabel('Number of pixels')
    plt.show()

p3, p80 = np.percentile(img, (3, 80))
img_rescale = exposure.rescale_intensity(img, in_range=(p3, p80))
show(img_rescale, 'Contrast stretch (3rd - 80th percentile)')
```

Result / Output



Observation

By plotting the rescaled image and its histogram, we can observe that the contrast has been enhanced. The histogram shows how the narrow range of original pixel values has been stretched to cover a wider range, resulting in a clearer image.

Task 2: Histogram Equalization

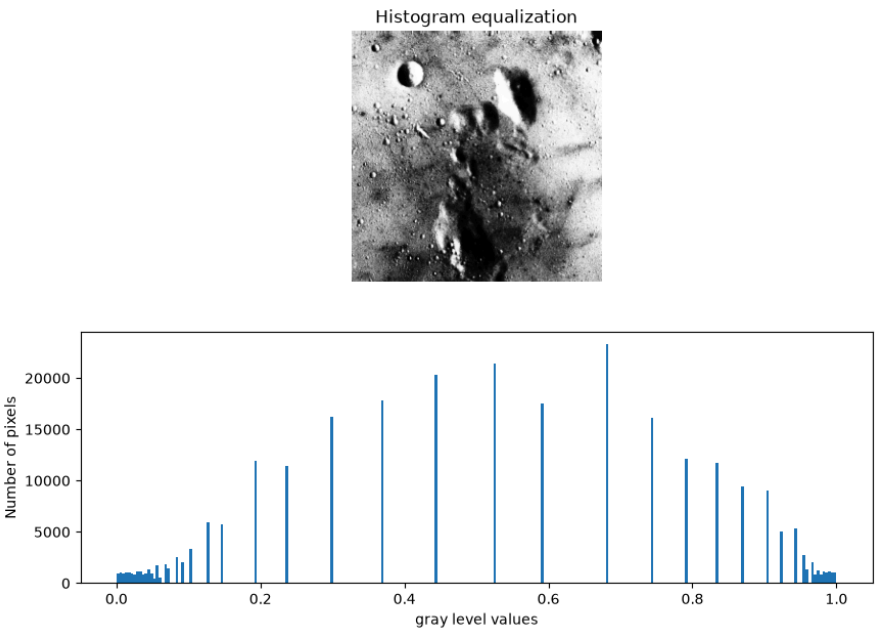
Description

Here, we applied histogram equalization to the 'moon' image using the `exposure.equalize_hist` function to improve global contrast.

Python Code

```
img_eq = exposure.equalize_hist(img)
show(img_eq, 'Histogram equalization', rng=(0, 1))
```

Result / Output Image



Observation

Histogram equalization effectively improved the global contrast of the image. By examining the plotted histogram, we can see that it has been flattened, meaning the pixel intensities are now distributed more uniformly across the available range.

Task 3: Histogram Matching

Description

In this final task, we implemented histogram matching using the 'chelsea' (cat) image as our base image and the 'rocket' image as the reference.

Python Code

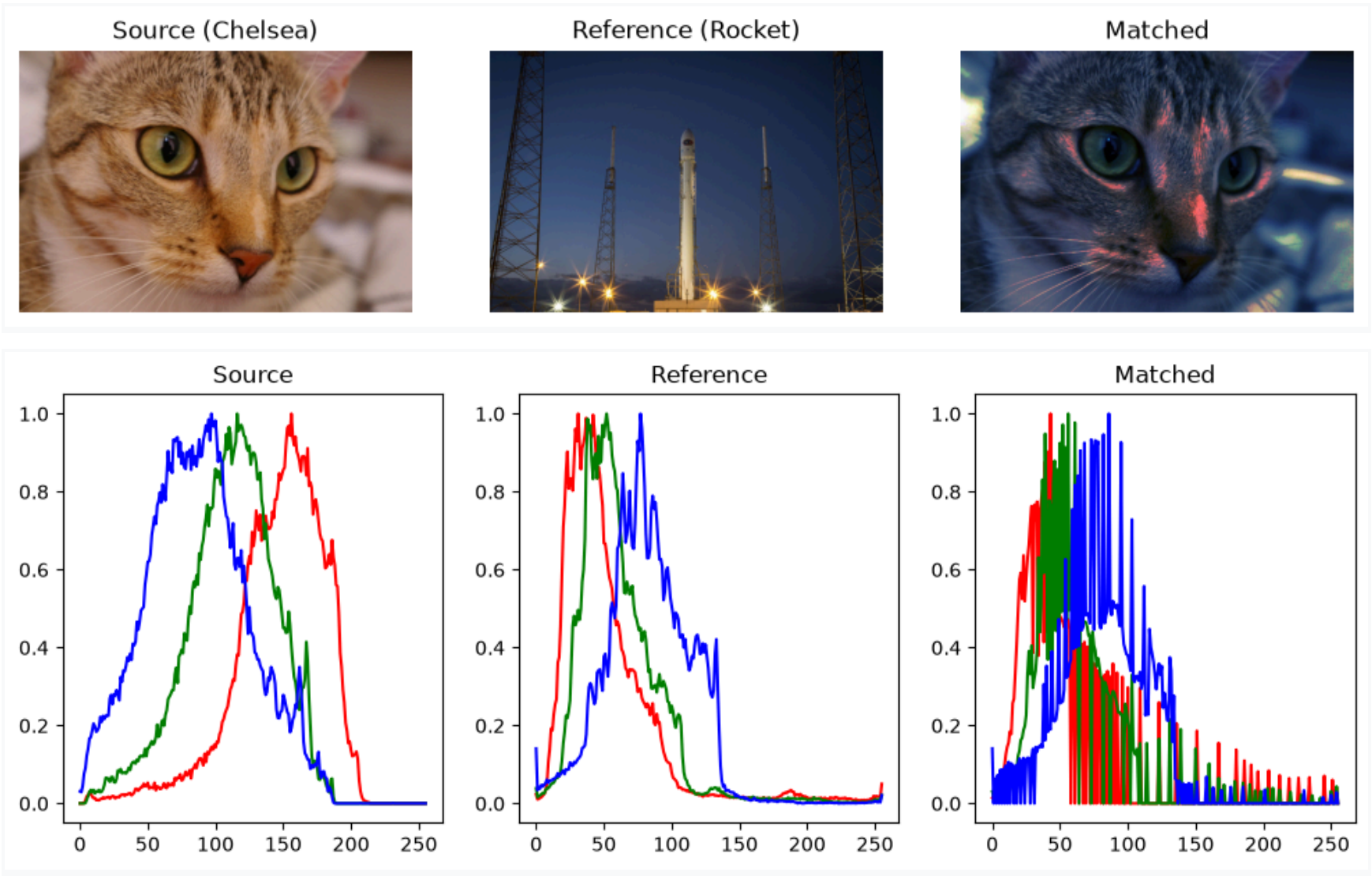
```
reference = data.rocket()
image = data.chelsea()

matched = exposure.match_histograms(image, reference, channel_axis=-1)

fig, axes = plt.subplots(1, 3, figsize=(12, 4))
for ax, im, t in zip(axes, [image, reference, matched],
                    ['Source (Chelsea)', 'Reference (Rocket)', 'Matched']):
    ax.imshow(im)
    ax.set_title(t)
    ax.axis('off')
plt.show()

fig, axes = plt.subplots(1, 3, figsize=(12, 4))
for ax, im, t in zip(axes, [image, reference, matched],
                    ['Source', 'Reference', 'Matched']):
    for i, c in enumerate(['red', 'green', 'blue']):
        h, bins = exposure.histogram(im[..., i], source_range='dtype')
        ax.plot(bins, h / h.max(), color=c)
    ax.set_title(t)
plt.show()
```

Result / Output Image



Observation

The resulting "Matched Image" successfully adopted the color characteristics and tone of the reference 'rocket' image. The overall color palette of the cat image shifted significantly to match the reference histogram.

Conclusion

Through this practical session, we successfully utilized Python to apply spatial domain image processing techniques. We practically demonstrated how intensity transformations and histogram manipulations (stretching, equalization, and matching) can profoundly impact visual quality and color representation in both grayscale and RGB images.